

术语详解 与工具指南

从编程环境到电路仿真

从嵌入式开发到 AI 辅助学习

13 个主题章节

定义 · 用法 · 官方入口

根据《微电子学习工具笔记》整理，面向初学者解释软件、编程语言、命令、文件格式和电子工程概念。保留基本操作、代码示例及可点击的官方资料链接。

使用方式：按课程与当前任务查阅，无需一次安装全部软件。新增示例用于教学，不代表原站作者的实验成果。

整理日期：2026-09-22

目录

点击章节名称或页码可跳转。正文中的青色链接通往官方网站或官方文档；PDF 书签提供章节与小节导航。

1. 开发工具、编程语言与运行环境	3
2. Git、GitHub 与命令语法	7
3. 编辑器、构建工具和嵌入式开发	9
4. 电路仿真与 RC 基础	12
5. 原理图、PCB 与制造文件	16
6. FPGA、HDL 与数字电路	18
7. 芯片、器件与多物理场仿真	21
8. 数学计算、Python 库与数据文件	23
9. 笔记、文献、报告与文件工具	26
10. 终端、远程连接与操作系统	28
11. 技术资料、博客与静态网站	30
12. AI 辅助学习和编程	33
13. 一套最小学习组合与综合练习	37
来源、范围与核对说明	39

先读第 1 章区分语言与工具；随后根据课程进入嵌入式、电路、PCB 或 FPGA 章节。第 13 章提供最小工具组合及 RC 综合练习。

1. 开发工具、编程语言与运行环境

1.1 先分清六个角色

名词	定义	例子与区别
源代码 (Source code)	人按编程语言规则编写的文本。	main.c、analysis.py。文件能打开不等于程序能运行。
编辑器 (Editor)	用于输入、查看、搜索和修改文本的软件。	VS Code 可以写 C、Python 和 Markdown，但不同语言还需要各自的运行工具。
编译器 (Compiler)	把源代码翻译为另一种代码的程序，常见目标是机器码或目标文件。	GCC、Clang、MSVC。它们还会检查许多语法和类型错误，但不会保证算法正确。
解释器 (Interpreter)	负责执行语言程序的运行系统。	常用 Python 实现 CPython 会先生成字节码，再由其虚拟机执行；“解释执行”不等于完全没有编译步骤。
调试器 (Debugger)	控制程序执行并观察内部状态的工具。	GDB 可以暂停、单步和检查变量。调试器不是代码编辑器。
IDE (Integrated Development Environment)	集成开发环境，把编辑、工程管理、构建和调试整合起来。	PyCharm、CLion、STM32CubeIDE。IDE 仍需选定或配套安装解释器、编译器及 SDK。

构建 (Build) 是从源文件得到可运行产物的过程，可能包括代码生成、编译、链接和资源打包。链接 (Link) 把多个目标文件与库组合起来，解析函数等符号的引用。工具链 (Toolchain) 指完成这些步骤的一组相互配合的工具。

例如：在 VS Code 写 main.c → GCC 编译 → 链接器生成程序 → 运行 → 用 GDB 定位问题。按下 IDE 的“运行”按钮，往往只是让 IDE 替你组织了这些步骤。[VS Code 的 C/C++ 官方说明](#)

1.2 C 与 C++：编译型通用语言

C侧重函数、数据类型、指针和对内存的直接操作，常用于单片机驱动和底层程序。C++在这些能力上提供类、模板、标准库等机制，适用于复杂软件及嵌入式工程。“C/C++”是把两种相关语言合称，不是一种独立语言；也并非所有 C 程序都能原样当作 C++ 编译。

推荐环境：初学者可用 VS Code 或 CLion；编译器按系统选择。若课程明确指定 Keil、STM32CubeIDE 或其他环境，优先沿用课程配置。

编译器或工具	定义与推荐场景	基本用法与官方入口
GCC / G++	GNU 编译器集合；gcc 常用于 C，g++ 常用于 C++，后者会默认处理 C++ 标准库链接。	编译命令见下例。 GCC 官网

编译器或工具	定义与推荐场景	基本用法与官方入口
Clang / LLVM	Clang 是 LLVM 项目中的 C/C++ 等语言前端及编译器驱动；LLVM 提供相关编译基础设施。	使用 clang 编译 C，clang++ 编译 C++。Clang 官网
MSVC	Microsoft Visual C++ 工具集，适合 Windows 原生开发。Visual Studio 是 IDE，MSVC 是其中的编译工具。	在配置好的开发者命令提示符或 IDE 中编译。MSVC 文档
MSYS2 / MinGW-w64	MSYS2 提供 Windows 开发环境与包管理；MinGW-w64 提供面向 Windows 的运行时、头文件等，常与 GCC 配合。二者不等于一种语言。	Windows 上按官方指南安装 UCRT64 等适合的环境，再让 VS Code 指向实际编译器。MSYS2 官网
Xcode Command Line Tools	Apple 提供的命令行开发工具包，包含 Clang 和 macOS SDK 等。	macOS 上可作为 VS Code 或其他编辑器的底层 C/C++ 工具链。Apple 安装说明

C 语法示例：保存为 main.c。

```
C
#include <stdio.h>

int main(void) {
    double resistance = 10000.0;
    double capacitance = 100e-9;
    double tau = resistance * capacitance;
    printf("tau = %.6f s\n", tau);
    return 0;
}
```

- `#include` 是预处理指令，声明要使用标准输入输出头文件。
- `main` 是这个普通桌面程序的入口函数；嵌入式程序通常先执行启动代码，再进入 `main`。
- `double` 是双精度浮点类型；`=` 是赋值，`*` 是乘法，`;` 结束语句，`{ }` 包围代码块。
- `100e-9` 表示 100×10^{-9} ，这里是 100 nF。
- `printf` 是格式化输出函数；`%.6f` 保留六位小数，`\n` 换行。
- `return 0` 表示这个程序正常结束。

安装 GCC 后，在文件所在目录执行：

```
SH
gcc -std=c11 -Wall -Wextra main.c -o rc_demo
```

`-std=c11` 选择语言标准；`-Wall -Wextra` 开启一组常用警告；`-o` 指定输出名称。在 macOS/Linux 执行 `./rc_demo`；Windows 可将输出命名为 `rc_demo.exe`，在 PowerShell 中执行 `./rc_demo.exe`。若使用 Clang，可把 `gcc` 换成 `clang`。

C++ 示例：保存为 main.cpp，使用 `g++ -std=c++17 -Wall -Wextra main.cpp -o cpp_demo` 编译。

CPP

```
#include <iostream>

int main() {
    const double tau = 10000.0 * 100e-9;
    std::cout << "tau = " << tau << " s\n";
}
```

`const` 表示这里的变量初始化后不能再被赋值；`std::` 指向标准库命名空间；此处 `<<` 是输出操作。编译单片机固件时不能直接照搬桌面编译命令，还需要目标芯片配置、启动文件和链接脚本。

1.3 Python：脚本、数据分析与自动化

定义：Python 是通用编程语言，适合处理 CSV、计算电路参数、绘制图形和自动化重复操作。[官方教程](#)

推荐环境：使用 [Python 官方发行版](#) 提供的 CPython；编辑器选择 VS Code，或者使用专门面向 Python 的 [PyCharm](#)。交互计算可用 Jupyter。一般入门任务不需要另外寻找“Python 编译器”。

PYTHON

```
import math

resistance = 10_000
capacitance = 100e-9
tau = resistance * capacitance
fc = 1 / (2 * math.pi * tau)
print(f"时间常数: {tau * 1000:.3f} ms")
print(f"截止频率: {fc:.2f} Hz")
```

保存为 rc.py，在选好的 Python 环境中执行 `python rc.py`；有的 macOS/Linux 环境命令名是 `python3`。

- `import` 导入模块；`math.pi` 是模块中的圆周率常量。
- `10_000` 中的下划线用于增强数字可读性。
- `f"..."` 是格式化字符串；`{fc:.2f}` 显示两位小数。
- Python 通常用缩进划分代码块；`if`、`for` 和函数定义后的冒号会引入代码块。
- `#` 开始注释；Python 注释和 C 的 `#include` 不是同一种机制。

1.4 MATLAB：既是语言，也是计算环境

定义：MATLAB 是以数组、矩阵和数值运算为核心的编程语言及开发环境。常见源文件后缀为 `.m`。它与 Python 的语法、索引规则不同。[MATLAB 官网](#)

推荐环境：入门直接使用 MATLAB 自带编辑器和命令窗口，不需要另装 GCC 来运行 .m 文件。MATLAB Compiler 是用于部署特定应用的产品，不是日常写 MATLAB 脚本的必需品。

```
MATLAB
R = 10e3;
C = 100e-9;
tau = R * C;
fc = 1 / (2 * pi * tau);
fprintf('fc = %.2f Hz\n', fc);
```

；常用于抑制当前语句结果的自动显示；% 开始注释；数组索引通常从 1 开始。* 表示矩阵乘法，.* 表示逐元素乘法，这个区别在处理多组数据时很重要。

1.5 其他“看起来像代码”的内容

名称	是什么	用什么编辑、运行或转换
Verilog / SystemVerilog	描述与验证数字硬件的语言，详见第 6 节。	VS Code 或厂商编辑器；用 HDL 仿真器验证，用综合工具生成电路。
Markdown	轻量标记语法，表达标题、列表、链接和代码块；不是通用编程语言。	VS Code、Obsidian 编辑和预览；不需要 C 编译器。 CommonMark 规范
LaTeX	文档排版系统及标记语法，适合公式和论文。	Overleaf 或本地编辑器；用 XeLaTeX、LuaLaTeX、pdfLaTeX 等排版引擎生成 PDF。 LaTeX 官网
HTML	描述网页结构的标记语言。	VS Code 编辑，浏览器显示。 HTML 标准
CSS (补充)	控制网页颜色、字体和布局的样式语言。	VS Code 编辑，由浏览器应用样式。 W3C CSS
JavaScript (补充)	常用于网页交互的编程语言。	VS Code 编辑；由浏览器 JavaScript 引擎或 Node.js 运行。
YAML / frontmatter	YAML 是数据表达格式；frontmatter 是文档头部的元数据区域，常用 YAML，也可能用 TOML 等格式。	编辑文本，由网站生成器解析；并不是编译器指令。
Shell 命令	告诉命令解释器运行程序、传递参数或操作文件。	Bash、PowerShell 等执行，不需要先编译成 C 程序。
SPICE 网表	用文本描述电路元件、节点、模型与分析指令。	文本编辑器编写，由 LTspice、ngspice 等仿真器读取。

Markdown 例子：# 实验报告 是一级标题，**结论** 表示加粗，[\[Git 官网\]\(https://git-scm.com/\)](https://git-scm.com/) 表示链接。LaTeX 例子：\tau = RC 表示公式中的希腊字母 τ ；完整文档需要文档类和正文环境，详见第 9 节。

2. Git、GitHub 与命令语法

2.1 工具与平台

名称与官方入口	定义	基本用法
Git	分布式版本控制软件，在本地记录文件版本及其关系。	在项目中初始化或克隆仓库；修改后查看差异、暂存并提交。
GitHub	托管 Git 仓库、组织协作与问题讨论的平台。	建仓库，上传提交，用 Issue 记录问题，通过 Pull Request 审查修改。
GitHub Desktop	Git 的图形客户端。	克隆仓库，在界面勾选文件、填写说明、创建提交和同步。
GitHub Pages	发布静态网站的托管服务。	准备站点文件，配置发布来源或构建工作流，再访问生成的网站。
Gitee	代码托管与协作平台。	操作思路与其他 Git 托管平台类似，远程仓库地址和平台功能不同。

Git 不要求每次提交都联网；GitHub 也不是 Git 的另一个名字。Git 版本历史不自动包含未跟踪文件，远端同步也不能替代所有原始数据的备份。[Git 入门文档](#)

2.2 版本控制中的名词

术语	定义与例子
仓库 (Repository / Repo)	保存项目文件及版本历史的容器。本地 Git 仓库通常有 .git 元数据目录。
工作区 (Working tree)	你实际打开、编辑的那份文件。
暂存区 (Index / Staging area)	准备进入下一次提交的内容。文件暂存后再次修改，新修改不会自动进入暂存区。
提交 (Commit)	保存一次项目快照，并记录说明、作者和父提交等信息。不是“上传”的同义词。
差异 (Diff)	两个版本或状态之间的修改，常显示新增行和删除行。
分支 (Branch)	指向某条开发历史位置的可移动引用，便于分别开发。
合并 (Merge)	将不同开发线的修改整合起来。
冲突 (Conflict)	Git 无法自动确定如何组合的修改，需要人工选择正确结果。
远端 (Remote)	本地仓库记录的其他仓库地址；origin 是常用名称。
Issue	平台中的问题、任务或改进建议记录。

术语	定义与例子
Pull Request (PR)	请求审查并合入一组修改的协作流程；与命令 <code>git pull</code> 不同。
README	项目的说明文件，常写目标、依赖、运行方式和目录结构。
.gitignore	描述哪些未跟踪文件通常应被忽略的规则文件。不会自动移除已经跟踪的文件。

2.3 原笔记中的 Git 命令逐条解释

这些是命令行语法。安装 Git 后，在终端进入项目目录执行即可；不需要编译器。`<file>` 和 `<url>` 是占位符，使用时应替换为实际路径或地址，通常不保留尖括号。

命令	含义	适合什么时候使用
<code>git status</code>	查看分支、已暂存、未暂存及未跟踪文件。	修改前后都可检查。
<code>git diff</code>	比较工作区与暂存区中已跟踪文件的差异。	检查还没暂存的修改；不会直接显示新建未跟踪文件的全文。
<code>git diff --staged</code>	比较暂存区与当前提交。	提交前确认本次到底包含什么。
<code>git log --oneline</code>	用一行一个提交的形式显示历史。	查找改动记录。
<code>git clone <url></code>	把远程仓库克隆到本地。	第一次获取已有项目。
<code>git add <file></code>	把文件当前的修改加入暂存区。	精确选择本次提交范围。
<code>git commit -m "说明"</code>	创建本地提交； <code>-m</code> 后面是提交说明。	一项修改完成并检查后。
<code>git fetch</code>	获取远端的新对象和远程跟踪分支信息。	先了解远程更新，通常不会直接改当前工作区。
<code>git pull</code>	获取更新，并按配置将其整合到当前分支，可能使用 <code>merge</code> 或 <code>rebase</code> 。	准备接受远程修改时。
<code>git push</code>	向远端发送提交并请求更新对应引用。	需要同步或分享本地提交时。

例如修改报告后：`git diff` → `git add report.md` → `git diff --staged` → `git commit -m "补充 RC 截止频率计算"`。这里的引号用于把带空格或文字的说明作为一个参数传入。

3. 编辑器、构建工具和嵌入式开发

3.1 编辑器与开发环境

工具与官方入口	定义、用途	简要上手方法
VS Code	可扩展的代码编辑器，能处理多种语言和 Markdown。	打开项目文件夹 → 安装对应语言扩展 → 选择解释器/编译器 → 运行与调试。
PyCharm	面向 Python 的 IDE，提供代码分析、运行配置和调试。	新建项目 → 选择虚拟环境 → 新建 .py 文件 → 运行；点击行号旁设置断点。
CLion	面向 C/C++ 的 IDE，可配合 CMake 和嵌入式工具链。	打开工程 → 配置工具链与 CMake → 构建 → 调试。编译器及调试器须与目标匹配。
STM32CubeIDE	ST 面向 STM32 的开发环境。官方提供基于 Eclipse 与 VS Code 的相关形态。	选择芯片或开发板 → 配置工程及外设 → 生成/编写代码 → 编译 → 连接探针下载调试。
Keil μVision	嵌入式 IDE；配合不同产品工具链支持不同处理器家族。	建工程选器件 → 安装器件支持包 → 配置编译与调试目标 → 编译和下载。8051 的 C51 工具链不能当作 STM32 的 Arm 工具链使用。
ESP-IDF	Espressif IoT Development Framework，乐鑫芯片的官方开发框架与配套工具。	安装匹配环境 → 选择示例 → 设置目标芯片 → 构建 → 烧录并监看串口。
Arduino IDE	用于 Arduino 生态开发板的编辑、构建和上传环境。	安装板卡支持包 → 选板卡与端口 → 打开 Blink 示例 → 验证编译 → 上传。底层工具链由板卡平台决定。
PlatformIO	管理嵌入式平台、框架、库和构建环境的工具生态。	创建项目选择开发板及框架 → 检查 platformio.ini → 构建、上传、串口监视。常与 VS Code 配合。

语言扩展/插件为编辑器添加补全、调试入口等能力；板卡支持包（Board package）提供板卡定义与相关工具；器件包（Device pack）提供芯片描述、启动文件或调试支持等。它们不完全是同一种东西。

3.2 构建、调试工具与硬件

名称与官方入口	定义	基本使用方式
Arm GNU Toolchain	面向 Arm 目标的 GNU 编译工具集合。	选匹配目标和主机的发行包，在 IDE/CMake 中配置编译器；裸机 Cortex-M 工程常见命令前缀为 arm-none-eabi-。

名称与官方入口	定义	基本使用方式
CMake	描述项目目标、依赖和构建规则，并生成或驱动底层构建系统的工具。	在 CMakeLists.txt 中定义项目，运行 <code>cmake -S . -B build</code> 配置，再运行 <code>cmake --build build</code> 。它不是 C/C++ 编译器。
Make	依据依赖关系和规则决定要执行哪些构建命令。	阅读 Makefile，用 <code>make</code> 构建默认目标或 <code>make 目标名</code> 构建指定目标。
Ninja	注重快速增量构建的工具。	常由 CMake 生成 <code>build.ninja</code> ，再由 Ninja 执行；初学时无需手写它的规则文件。
GDB	GNU 调试器，可调试本地程序或连接远程目标。	加载含调试信息的程序，设置断点、运行、单步和查看变量；嵌入式常经调试服务连接硬件。
OpenOCD	连接调试探针与目标芯片的软件，可提供 GDB 服务及烧录能力。	选择正确的探针和芯片配置，启动服务，再让 GDB/IDE 连接。
J-Link GDB Server	SEGGER 提供的 GDB 与 J-Link 探针之间的调试服务。	指定目标器件和 SWD/JTAG 等接口，启动后配置 IDE 的远程调试连接。
ST-LINK	ST 的硬件调试/编程探针家族。	通过匹配的接口连接目标板，确认参考电压与地，再由开发环境访问芯片。链接为家族中的一款产品。
J-Link	SEGGER 的硬件调试探针家族。	选择匹配的型号和连接方式，在 IDE 中选择 J-Link 调试器。
VOFA+	可接收数据并显示曲线等内容的上位机软件。	选择串口和通信参数，选匹配的数据协议，将固件输出的数据映射为曲线通道。

典型的嵌入式调试连接是：IDE → GDB → OpenOCD 或 J-Link GDB Server → 调试探针 → 芯片。厂商环境可能使用其他调试服务，因此不要求每个工程都安装这条链上的全部软件。

3.3 芯片、接口和调试术语

术语	定义与在工程中的含义
MCU / 单片机	Microcontroller Unit，将处理器核心、存储器及外设集成在一颗芯片上的器件。
STM32	ST 的 32 位微控制器家族，具体外设、内核和引脚依型号而不同。
ESP32	乐鑫的芯片系列名称，不是单一内核型号；不同型号可能使用 Xtensa 或 RISC-V。
Arm / Cortex-M	Arm 是处理器架构与 IP 技术提供方；Cortex-M 是面向微控制器的处理器核心系列。
Xtensa / RISC-V	指令集或处理器架构相关体系。程序必须编译为目标处理器支持的指令。
8051	经典微控制器架构及其兼容衍生系列，常见于入门课程。

术语	定义与在工程中的含义
交叉编译	在一种主机环境中生成供另一种目标环境运行的程序，例如在电脑上编译 STM32 固件。
固件 (Firmware)	在设备中执行、控制硬件行为的软件，常存储于 Flash 等非易失存储器。
SDK	Software Development Kit，软件开发工具包，通常包含接口、库、头文件、示例和工具。
烧录 / 下载	将程序写入目标设备存储器；这里“下载”不一定指从互联网下载。
SWD	Serial Wire Debug，常用于 Arm 微控制器的调试接口，核心信号包括 SWDIO 与 SWCLK。
JTAG	用于测试及调试的一类标准接口，通常使用 TCK、TMS、TDI、TDO 等信号。
串口 / UART	UART 是通用异步收发器。串口通信需匹配波特率、数据位、校验和停止位，并确认电平标准。USB 连接不必然就是 UART。
波特率	每秒传输的符号数；常见二进制 UART 中与比特率数值相同，但有效数据吞吐还受起止位等影响。
GPIO	通用输入输出引脚，可读取或输出数字电平；是否支持复用功能取决于芯片。
外设 (Peripheral)	芯片内或外部的功能模块，例如定时器、UART、ADC。
寄存器 (Register)	保存控制或状态信息的存储单元。外设寄存器中的不同位可能控制启动、模式或中断标志。
时钟 (Clock)	提供操作节奏的周期信号。外设频率、波特率和定时精度都可能依赖它。
断点 (Breakpoint)	让程序运行到指定位置或满足条件时暂停。
单步 (Step)	一步一步执行代码；“步入”进入函数，“步过”把函数调用整体执行完。
调用栈 (Call stack)	当前函数调用关系的记录，帮助定位“从哪里调用到了这里”。
日志 (Log)	程序输出的事件或状态记录；串口日志常用于观察程序执行过程。
API / API Key	API 是软件调用接口；API Key 是部分服务使用的访问凭据，两者不能混为一谈。

ESP32 的具体配置应在官方文档中切换到相应芯片型号，而不是把某一型号教程套用到整个系列。[ESP-IDF 入门文档](#)

4. 电路仿真与 RC 基础

4.1 仿真软件及基本操作

仿真 (Simulation) 是用数学模型计算系统在给定条件下的行为。输入模型、参数和激励，得到电压、电流等结果；结果反映模型与条件，不自动等于实物性能。

SPICE最初是 Simulation Program with Integrated Circuit Emphasis 的缩写，现在也常用来指相关电路仿真方法、程序及网表生态。不同 SPICE 软件的方言、模型和指令兼容性可能不同。

软件与官方入口	定义与用途	基本用法
LTspice	Analog Devices 提供的 SPICE 仿真软件，适合模拟电路与电源分析。	放置器件、接线及参考地 → 填参数和激励 → 设分析类型 → 运行 → 点击节点观察电压或器件观察电流。
Multisim	图形化电路设计与 SPICE 仿真环境，常用于教学。	搭电路 → 接虚拟信号源和示波器 → 运行；也可使用分析功能获得频响等结果。
ngspice	开源 SPICE 电路仿真器，便于命令行、脚本或其他软件集成。	编写网表并加载，执行分析指令，用图形命令或导出数据查看结果。
Proteus	包含电路设计和嵌入式系统仿真等功能的工具套件。	选受支持的 MCU 模型 → 连接外部电路 → 载入对应固件 → 设置时钟 → 运行仿真。须确认该器件及外设模型是否受支持。

4.2 原理图、网表、模型和分析方式

术语	定义	用 RC 电路理解
原理图 (Schematic)	用符号表达元件及电气连接的图。	画出输入源、电阻、电容以及地。
节点 (Node)	理想导线直接连接、具有同一电位的一组连接点。	输入节点 in、输出节点 out、参考节点 0。
网表 (Netlist)	用文本列出器件、连接节点、参数或模型。	R1 in out 10k 表示 10 kΩ 电阻连接输入与输出。
器件模型 (Model)	对器件行为的数学描述。	理想电容只描述电容效应，实际模型还可能包括串联电阻等。
激励 (Excitation)	施加给系统的输入或外部作用。	阶跃电压、正弦信号或脉冲。
工作点 / DC Operating Point	在规定直流条件下求静态电压、电流和器件工作状态。	稳态直流下理想电容不传导直流电流。
瞬态分析 / Transient	计算随时间变化的响应。	看电容充电曲线和输入脉冲经过滤波后的形状。

术语	定义	用 RC 电路理解
AC 小信号分析	在工作点附近线性化电路，计算不同频率下的小信号响应。	绘制输出/输入的幅值与相位，寻找截止频率。它不是大信号正弦瞬态扫描的同义词。
参数扫描 / Sweep	在多个参数值下重复计算。	把电容改成 47 nF、100 nF、220 nF，比较截止频率。
偏置 (Bias)	为有源器件建立所需直流工作状态。	放大器需要合适偏置，才能围绕工作点处理输入信号。
小信号模型	工作点附近的局部线性近似。	输入变化足够小时，用增量电压、电流分析增益。
饱和	含义取决于器件：BJT 饱和、MOSFET 饱和区、运放输出饱和不是同一个概念。	MOSFET 饱和区常用于模拟放大；运放输出饱和则可能意味着输出接近摆幅极限并失真。
寄生参数	实际结构附带、但理想原理图可能没有画出的电阻、电容和电感。	导线有电感，电容存在等效串联电阻。
过冲 / 振荡	过冲是响应暂时超过最终值；振荡是量随时间来回变化。	理想单级无源 RC 对阶跃的响应单调；实测过冲需要检查其他环节、寄生与测量方式。

4.3 RC、时间常数和截止频率

R代表电阻，单位 Ω ；C代表电容，单位 F。k 表示 10^3 ，m 表示 10^{-3} ， μ 表示 10^{-6} ，n 表示 10^{-9} 。注意毫秒 ms 和秒 s 相差 1000 倍。

RC 低通滤波器：输入经过串联电阻到输出，输出节点通过电容接参考地；输出取电容两端。低频成分较容易通过，高频成分衰减更大。输出取在不同元件上会改变电路功能。

在理想输入源、输出近似无负载、器件理想的条件下：

TEXT

时间常数： $\tau = RC$
截止频率： $f_c = 1 / (2\pi RC)$
传递函数： $H(j\omega) = V_{out} / V_{in} = 1 / (1 + j\omega RC)$
 $\omega = 2\pi f, j^2 = -1$

取 $R = 10 \text{ k}\Omega$ 、 $C = 100 \text{ nF}$ ，得到 $\tau = 0.001 \text{ s} = 1 \text{ ms}$ ， $f_c \approx 159.15 \text{ Hz}$ 。

- 时间常数 τ ：刻画响应快慢。电容初始电压为零、输入从零阶跃到 V 时， $v_{out}(t) = V(1 - e^{-(t/\tau)})$ ；经过一个 τ 到达最终值约 63.2%，五个 τ 到达约 99.3%。
- 频率 f：每秒周期数，单位 Hz。角频率 ω 单位 rad/s。
- 增益 (Gain)：输出量与输入量的比值。电压增益分贝值通常写为 $20 \log_{10}(|V_{out}/V_{in}|)$ 。
- 相位 (Phase)：周期信号在时间上的相对位置，常用角度表示。理想 RC 低通在截止频率处相位为 -45° 。
- 截止频率：这里指幅值降到低频值的 $1/\sqrt{2} \approx 0.707$ ，即约 -3.01 dB 处，不表示更高频率完全无法通过。

- 带宽 (Bandwidth)：满足指定响应标准的频率范围。理想单极点低通的常用 -3 dB 带宽为从直流到 f_c 。
- 频响 (Frequency response)：系统对不同频率的增益与相位响应。波特图 (Bode plot) 常用对数频率轴表示它。
- 滤波 (Filtering)：按频率或其他规律改变信号成分。RC 低通常用于平滑波形或衰减高频噪声。

若信号源有内阻、后级输入阻抗不够高，或 ADC 采样电容影响较大，实际电路就需要纳入这些因素重新分析。

4.4 SPICE 网表中的“语法”

下面是一个用于说明语法的简单网表；不同仿真器的显示和导出命令另行设置：

```
SPICE
RC low-pass example
V1 in 0 DC 0 AC 1 PULSE(0 1 0 1u 1u 10m 20m)
R1 in out 10k
C1 out 0 100n
.op
.ac dec 100 1 100k
.tran 10u 10m
.end
```

- 第一行为标题；V1 为电压源，R1 为电阻，C1 为电容。
- 0 是参考地节点；in 和 out 是自定义节点名。
- AC 1 设置小信号 AC 激励幅值；PULSE(...) 设置瞬态脉冲激励，两者用于不同分析。
- .ac dec 100 1 100k 表示每十倍频程取 100 点，从 1 Hz 扫描至 100 kHz。
- .tran 10u 10m 设置瞬态分析参数；具体输出步长与内部自适应步长的含义应以仿真器文档为准。
- SPICE 中 m 通常表示毫，兆常写 meg；不要凭大写 M 就认定是兆。

编辑器可用 VS Code，也可以在 LTspice 中绘制原理图后让软件生成网表。它由 SPICE 仿真器解释，不交给 GCC。[ngspice 官方手册入口](#)

4.5 实验仪器与采样

术语	定义与使用要点
面包板	无需焊接即可插接元件的实验板。先确认内部导通分组；不适合把任何高频或大电流电路都直接搬上去。
信号源	产生正弦、方波等测试信号的仪器；需设置幅度、频率、偏置及负载相关参数。
示波器	显示电压随时间变化的仪器。设置通道量程、时间刻度、触发和探头倍率。
输入幅值 / 直流偏置	幅值描述变化大小；偏置描述整体电平的位置。例如 1 V 峰值加 1.5 V 偏置，对应约 0.5~2.5 V。注意区分峰值和峰峰值。

术语	定义与使用要点
ADC	Analog-to-Digital Converter，模数转换器，将输入模拟量转换为数字码。分辨率、参考电压、采样率及输入范围共同影响结果。
采样率 f_s	每秒采集的样本数。对于带限信号，理想重建要求采样率大于最高频率的两倍；实际还需抗混叠滤波和余量。
量化	把连续幅度映射为有限个数字级别；会引入量化误差。
共地	非隔离测量/通信系统共享正确的电位参考。台式示波器地夹常与保护地相连，不能把它任意夹在非地节点。
限压 / 限流	控制输入电压和电流不超出器件范围；设限值前要知道器件额定值和接口允许条件。
实测 / 理论 / 仿真	实测来自仪器；理论来自解析模型；仿真来自数值模型。这三类数据应明确标注。

5. 原理图、PCB 与制造文件

5.1 三种常见设计软件

软件与官方入口	定义	基本用法
嘉立创 EDA	提供原理图、PCB 和制造相关工作流的 EDA 工具。	新建工程 → 放置器件和连接 → 分配封装 → 转 PCB → 规则检查 → 导出制造文件。 专业版入门
Altium Designer	集成原理图、PCB、库与设计规则管理的电子设计环境。	建工程与文档 → 配置库和规则 → 同步原理图到 PCB → 布局布线 → 生成输出。
KiCad	开源电子设计套件，涵盖原理图和 PCB 编辑等功能。	创建项目 → 画原理图并检查 ERC → 分配封装 → 更新 PCB → 布线和 DRC → 导出文件。 官方文档

5.2 PCB 术语逐项解释

术语	定义与例子
EDA	Electronic Design Automation，电子设计自动化，是一大类工具与方法；“嘉立创 EDA”是其中的具体产品名称。
PCB	Printed Circuit Board，印制电路板，通过铜箔走线、焊盘与过孔连接器件。
原理图符号 (Symbol)	表示器件逻辑/电气功能与引脚的图形。一个器件符号不等于其真实外形。
引脚 (Pin)	器件与外部连接的端子；引脚编号必须与实际器件及封装对应。
PCB 封装 / Footprint	PCB 上焊盘、丝印、外形等几何定义，与器件的实际引脚布局配合。
器件物理封装 / Package	芯片或元件实际的外壳与引脚结构，例如 QFN、SOIC。它与软件中的 footprint 相关，但不是同一个对象。
焊盘 (Pad)	PCB 上用于焊接或接触的导电区域。
网络 (Net)	应保持电气连接的一组端点，例如 GND、3V3。
布局 (Placement)	安排元件在板上的位置和方向。
布线 (Routing)	用铜走线等实现电气网络连接。
过孔 (Via)	连接不同铜层的金属化孔或其他层间连接结构。
板框 / 层叠	板框定义电路板边界；层叠定义铜层、介质层及其厚度和排列。
器件库	保存符号、封装、参数或模型等资料的集合。应检查具体库中这些内容是否完整、匹配。

术语	定义与例子
设计规则	对线宽、间距、过孔、差分对等设定的约束，要考虑制造能力与电气需求。
ERC	Electrical Rules Check，电气规则检查，例如检查某些未连接引脚和冲突连接类型。
DRC	Design Rules Check，设计规则检查，例如检查铜间距和线宽。工具只能检查已定义、可表达的规则。
去耦 (Decoupling)	通常在电源附近用电容等降低局部供电波动和噪声耦合；元件位置与电流回路同样重要。
回流路径 (Return path)	电流返回源端的路径。信号电流总要闭合，高频回流分布与参考平面、阻抗有关。
电平兼容	两个接口的输出电压、输入阈值及耐压条件彼此适配；不能只因为插头能接上就认定兼容。
BOM	Bill of Materials，物料清单，常包括位号、数量、参数、封装和采购型号。
Gerber	用于表达 PCB 各层图形等制造信息的一类标准文件。不是完整可编辑工程的替代品。 格式维护方
钻孔文件	给出孔位、孔径及相关加工信息的文件，常与 Gerber 一起提供。
装配文件 / 坐标文件	描述器件放置位置、角度与所在面的文件，供贴装等工序使用；格式按制造商要求。
打样	小批量制作样板，用来验证设计与装配。
额定值	器件在规定条件下可承受或工作的参数范围，例如电容耐压、电阻功率。

最常见的误区：原理图连接正确，不保证封装引脚正确；DRC 全通过，不保证供电设计、散热或功能正确。

6. FPGA、HDL 与数字电路

6.1 基本概念

FPGA（Field-Programmable Gate Array）是现场可编程门阵列。你主要是在配置可编程逻辑与连接关系，而不只是让固定 CPU 顺序执行普通软件。FPGA 中也可以包含或构建处理器，但这不是所有 FPGA 设计的必需部分。

HDL（Hardware Description Language）是硬件描述语言的统称。Verilog与 SystemVerilog用于描述与验证数字硬件；SystemVerilog 增加了类型、设计结构和验证能力。

RTL（Register Transfer Level，补充术语）是寄存器传输级，描述寄存器之间的数据传输及组合逻辑。不是所有合法 HDL 语句都能综合成目标硬件。

术语	定义
组合逻辑	在理想逻辑模型中，输出由当前输入决定，例如与门、加法器。实际器件存在传播延迟。
时序逻辑	输出或后续行为还依赖保存的状态，例如触发器、计数器。
时钟 / 复位	时钟协调状态更新；复位把状态带到指定初始值。复位可能同步或异步、有效电平也可能不同。
Testbench	测试平台代码，用来驱动待测模块、施加激励和检查输出；通常不作为上板电路的一部分综合。
功能仿真	在仿真器中检查 HDL 逻辑在测试激励下是否符合预期。只覆盖实际执行到的测试场景。
综合 (Synthesis)	把可综合 HDL 等描述转化为逻辑网表。
布局布线 / 实现	把逻辑映射到 FPGA 实际资源，并安排连接。与 PCB 布线对象不同。
时序约束	声明时钟、输入输出时序及特定路径条件等，告诉工具需要满足什么时间要求。
静态时序分析 (STA)	在给定约束和模型下计算路径延迟并检查时序，不需要像功能仿真那样枚举输入波形。
建立时间 / 保持时间	数据在触发器采样边沿前后需要稳定的时间要求。
时序报告	展示路径延迟、要求和裕量等结果，用来判断是否满足时序。
比特流 (Bitstream)	配置 FPGA 资源的文件。它与 CPU 机器指令程序不是同一类产物。
上板验证	把设计配置到实际开发板，检查引脚、电平、时钟和真实外设下的行为。

6.2 用什么编辑、仿真与综合

工具与官方入口	定义	简要用法
Quartus Prime	面向 Altera FPGA 等器件的设计工具。原笔记的 Altera/Intel 名称反映其品牌历史。	选择器件 → 添加 HDL → 设引脚及时序约束 → 综合与实现 → 分析时序 → 配置器件。版本要支持具体器件。
Vivado	AMD 面向相关 FPGA/自适应器件的开发环境。	创建 RTL 工程 → 选器件 → 仿真 → 综合 → 实现 → 生成比特流 → 连接硬件下载。不能假定支持所有历史 Xilinx 器件。
ModelSim / Questa	Siemens EDA 生态中的 HDL 仿真与验证产品名称；课程中也可能使用厂商配套版本。	建工作库 → 编译 HDL 与 testbench → 加载顶层 → 运行 → 看波形和断言结果。具体命令及功能按安装版本。
Icarus Verilog	开源 Verilog 编译与仿真工具。	用 iverilog 编译仿真模型，再用 vvp 执行；适合基础 Verilog 练习，SystemVerilog 支持需按功能确认。
Verilator	将受支持的 Verilog/SystemVerilog 转换为可编译仿真模型的工具，也提供 lint 等检查。	使用 lint 检查代码，或生成并构建仿真模型；具体时序/语言特性按版本文档配置。
GTKWave	波形文件查看器。	打开 VCD 等波形文件，添加信号，放大时间轴和比较跳变。它本身不负责生成仿真结果。

推荐组合：学基础语法可使用 VS Code + Icarus Verilog + GTKWave；做课程 FPGA 工程使用课程指定的 Quartus 或 Vivado 及匹配仿真器。这里的“编译 HDL”可能指生成仿真模型，也可能是厂商工具对整个实现流程的称呼，需要看语境。

6.3 一个能看懂的 Verilog 例子

文件 and_gate.v：

VERILOG

```
module and_gate(  
    input wire a,  
    input wire b,  
    output wire y  
);  
    assign y = a & b;  
endmodule
```

module 定义硬件模块；input/output 声明输入输出；wire 表示连线类型；assign 是连续赋值；& 是按位与。它描述输出始终由两个输入的与运算决定，不是 CPU 只执行一次的赋值。

配套测试文件 tb.v :

```
VERILOG
`timescale 1ns/1ps
module tb;
    reg a, b;
    wire y;
    and_gate dut(.a(a), .b(b), .y(y));

    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, tb);
        a = 0; b = 0;
        #10 a = 1;
        #10 b = 1;
        #10 a = 0;
        #10 $finish;
    end
endmodule
```

dut 是待测模块实例名；.a(a) 是按端口名连接；此处 reg 表示可在过程块赋值的变量，并不保证一定综合成寄存器。initial 在仿真开始时启动一个过程；#10 推进仿真时间；\$dumpfile/\$dumpvars 输出波形；\$finish 结束仿真。此处时间单位为 1 ns，精度为 1 ps。

安装相应工具后，在这两个文件所在目录依次运行：

```
SH
iverilog -g2012 -s tb -o sim.vvp and_gate.v tb.v
vvp sim.vvp
gtkwave wave.vcd
```

这组示例演示如何生成和查看波形；还没有加入自动判断成功或失败的断言，因此“仿真结束”不等于“自动验证通过”。本指南未实际执行 HDL 仿真。[Icarus 仿真文档](#)

7. 芯片、器件与多物理场仿真

7.1 方向与术语

术语	定义与区别
IC	Integrated Circuit，集成电路，把大量器件及互连集成在芯片中。
模拟 IC / 数字 IC	模拟 IC 处理连续电压电流等量；数字 IC 主要按离散逻辑状态处理信息。混合信号芯片可以包含两者。
MOS / MOSFET	MOS 指金属-氧化物-半导体相关结构；MOSFET 是利用栅电场控制沟道导电的晶体管。现代材料可能不再严格对应名称中的每个字。
TCAD	Technology Computer-Aided Design，工艺与器件层面的计算机辅助设计。可模拟掺杂、结构、载流子输运等物理过程。
PDK	Process Design Kit，工艺设计套件，常包含器件模型、层定义、设计规则和验证相关文件。与特定工艺及工具流程配套。
芯片版图 (Layout)	表示集成电路各工艺层几何形状的设计数据，例如扩散区、多晶硅和金属互连。不同于 PCB 器件布局。
开放 PDK	按相应许可公开的工艺设计资料集合；“开放”不自动意味着任何用途都不受条件约束，或已经获得流片资格。
物理设计	数字 IC 中把逻辑网表转换为实际几何实现的流程，包括布局、时钟树、布线等。
射频 (RF)	Radio Frequency，涉及较高频率电磁信号的技术领域；分析时经常需要考虑传输线、阻抗匹配和电磁耦合。
传输线	不能忽略信号传播时间和分布参数的连接结构，例如特定条件下的 PCB 走线。
边界条件	给模型边界指定的物理约束，例如电势、温度、固定支撑或电磁辐射条件。
网格 (Mesh)	把连续空间划分为计算单元，用于数值求解。网格过粗可能漏掉重要变化，过细则增加资源开销。
有限元分析 (FEA)	将问题离散为有限个单元后近似求解的一类数值方法。
多物理场	联合考虑不同物理过程的相互作用，例如电流产生热量，温度再改变材料参数。
数值验证 / 收敛性	检查结果是否随网格、时间步长、求解精度等改变而趋于稳定；求解器显示“完成”还不够。

7.2 软件及入门方式

名称与官方入口	定义与用途	简要用法
Cadence Virtuoso	定制 IC、模拟/RF 原理图与版图设计环境。	在课程配置的 PDK 中建库和单元 → 绘制电路 → 配测试平台 → 仿真 → 设计版图及验证。Cadence 是公司名，不等于单一软件。
Spectre	Cadence 的电路仿真产品系列，常与 Virtuoso 配合。	设置器件模型、激励和分析，运行仿真并观察结果；它承担电路求解角色。
HSPICE	Synopsys 的电路仿真工具，常用于晶体管级分析。	准备网表与模型 → 设分析指令 → 运行 → 读取结果或后处理。
Sentaurus TCAD	Synopsys 的半导体工艺与器件仿真工具系列。	定义结构、材料与掺杂 → 生成网格 → 选择物理模型 → 扫描偏压 → 分析电流、电场等。
Silvaco TCAD	Silvaco 公司的工艺/器件仿真产品系列；Silvaco 本身是公司名。	选课程指定的具体模块，建立工艺或器件模型，设置求解与提取参数。
KLayout	集成电路版图查看与编辑工具，也支持脚本化操作等功能。	打开 GDS/OASIS 文件 → 配置图层显示 → 查看或编辑几何 → 根据实际规则进行检查。不是 TCAD 求解器。
HFSS	高频三维电磁场仿真软件，常用于天线、射频结构与互连。	建几何 → 指定材料 → 设置端口和边界 → 划分网格并求解 → 查看场分布和 S 参数。
Ansys	工程仿真产品家族/品牌，覆盖结构、流体、热、电磁等方向。	按任务选择具体求解器，再做几何、材料、载荷、网格和结果分析。不能把“Ansys”理解成只有一个固定用途的软件。
Abaqus	SIMULIA 的有限元分析工具，常用于结构、非线性和相关耦合分析。	建几何和材料 → 定义接触、载荷和边界 → 网格 → 提交求解 → 后处理。

S 参数（补充）用端口入射波与反射/传输波的关系描述网络，常见 S11 表示某端口反射，S21 表示由端口 1 到端口 2 的传输。GDS/OASIS（补充）是常见芯片版图数据格式，不是 PCB Gerber 的别名。

这些工具通常需要先具备器件物理、电路或场论基础。学习目标应是建立正确模型并解释结果，而不只是完成安装。
[Synopsys TCAD 说明](#)、[Silvaco TCAD 说明](#)

8. 数学计算、Python 库与数据文件

8.1 计算软件与库

库（Library）是可被其他程序调用的功能集合；包（Package）通常是组织和分发代码的一种单位。NumPy 等不是另一个独立的 Python 解释器，而是在 Python 环境中导入使用的库。

工具与官方入口	定义	简要用法
MATLAB	数组、矩阵和工程计算环境，语言说明见第 1 节。	导入数据 → 用脚本计算 → 绘图 → 保存脚本与结果。
Simulink	基于模块图的建模与仿真环境，常用于动态系统和控制。	拖入信号源、系统模块和显示模块 → 连线 → 设参数、求解器与仿真时间 → 运行。
NumPy	Python 数组与数值运算基础库。	<code>import numpy as np</code> ，用数组统一计算多组电压、电流或时间数据。
SciPy	在 NumPy 基础上提供优化、积分、信号处理等科学计算功能。	从对应模块导入函数，例如用 <code>scipy.signal</code> 设计或分析滤波器。
Matplotlib	Python 绘图库。	创建图和坐标轴 → 绘曲线 → 设置坐标名、单位与图例 → 保存 PNG/PDF 等。
pandas	Python 表格数据处理库，核心对象包括 DataFrame。	读取 CSV → 检查字段与类型 → 处理缺失或重复记录 → 统计或导出。
Jupyter	在浏览器等界面中组织代码、文字、公式与输出的交互计算生态。	创建 Notebook → 选择内核 → 按顺序执行单元格 → 保存 .ipynb。
Excel	电子表格软件。	导入数据 → 核对列格式和单位 → 使用公式或图表 → 保存工作簿。大批量重复分析宜保留脚本以便复现。

DataFrame可理解为带行列标签的数据表。内核（Kernel）是执行 Notebook 代码并保存变量状态的进程。单元格（Cell）是 Notebook 中的一个代码或文字块。乱序执行可能让结果依赖不可见的历史状态，因此交付前应重启内核并从头运行。[Jupyter 官网](#)

8.2 解释器、环境与依赖管理

名词与入口	定义	基本用法
虚拟环境	给项目隔离一套 Python 包和环境路径，减少项目间依赖冲突。	在编辑器中选中该环境的解释器，不要只创建环境却仍运行系统 Python。
venv	Python 标准库提供的虚拟环境模块。	<code>python -m venv .venv</code> 创建环境，然后使用环境内的 Python。

名词与入口	定义	基本用法
pip	Python 包安装工具。	<code>python -m pip install 包名</code> ，这种写法把 pip 绑定到指定的 Python。
conda	环境与包管理工具，可管理 Python 及其他软件包。	创建指定版本环境、激活环境并安装依赖；已有课程工程优先沿用其环境文件。
uv	Python 项目、包和环境管理工具。	可用 <code>uv init</code> 初始化项目、 <code>uv add</code> 添加依赖、 <code>uv run</code> 执行；也提供 <code>uv venv</code> 等方式。
依赖 (Dependency)	程序运行或构建需要的外部包、库或工具。	例如绘图脚本依赖 Matplotlib；应记录版本而不只写“安装 Python”。
依赖清单 / 锁文件	清单表达需要哪些依赖；锁文件记录更具体的解析结果，帮助复现。	常见名称有 <code>requirements.txt</code> 、 <code>pyproject.toml</code> 、 <code>uv.lock</code> ，实际作用依工具和写法而定。

在新练习目录中创建环境后，可以直接调用它而无需先修改 Shell 激活策略：

```
SH
# macOS / Linux
python3 -m venv .venv
.venv/bin/python -m pip install numpy scipy matplotlib pandas
.venv/bin/python rc.py
```

Windows PowerShell 对应示例：

```
POWERSHELL
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install numpy scipy matplotlib pandas
.\.venv\Scripts\python.exe rc.py
```

这两组是供学习时执行的安装示例，本次整理没有替你安装这些依赖。若已用 conda 或 uv 管理该项目，应选用已有流程，不必叠加多套环境管理方式。

8.3 文件格式与数据分析术语

术语	定义与注意点
CSV	Comma-Separated Values，以分隔符组织表格的文本格式。需要确认分隔符、编码、表头、引号和单位。
字段 (Field)	一项数据的名称或含义，例如 <code>time_ms</code> 与 <code>voltage_v</code> 。这些名称是约定，不是 Python 关键字。
缺失值 / 非数值	缺失值表示数据缺少；非数值包括出现在数字列里的文本等异常内容。不可未经分析就统一填零。

术语	定义与注意点
重复记录 / 时间顺序	重复行可能来自重复采集或合法重复；时间倒序可能来自重启、缓冲或文件拼接。应先查原因。
拟合 (Fitting)	调整模型参数，使模型与数据尽量一致。例如由充电曲线拟合 τ 。拟合好不等于模型一定正确。
时域 / 频域	时域观察随时间变化；频域观察不同频率的组成或响应。
FFT (补充)	快速计算离散傅里叶变换的算法。频谱解释还需要采样率、记录长度、窗函数和幅度标定。
平均值 / 峰值	平均值表示整体水平；峰值表示最大幅度相关量，需说明是最大值、绝对峰值还是峰峰值。
原始数据 / 后处理	原始数据是采集得到的记录；后处理是清洗、计算、绘图等步骤。建议保留原始数据另生成结果。
PNG / PDF	PNG 是栅格图像格式；PDF 是页面文档格式，可含矢量和栅格内容。导出 PDF 不保证里面的图一定是矢量。
二进制文件	不是以普通可读文本组织的文件。许多工程文件和图片属于此类，Git 可以保存，但逐行审查差异通常较困难。
缓存 / 临时文件 / 生成目录	由软件运行产生的中间或可再生数据。判断是否忽略时要确认其确实能重新生成。

CSV 示例：

```
CSV
time_ms,voltage_v
0,0
1,0.6321
2,0.8647
```

上面是 1 V 理想阶跃下 RC 响应的演示数值，不是测量记录。`data/example.csv` 表示 `data` 文件夹中的文件；`figures/voltage.png` 表示结果图路径。相对路径通常相对于当前工作目录解释，而不总是相对于脚本所在位置。

9. 笔记、文献、报告与文件工具

9.1 学习资料和任务管理

工具与官方入口	定义	简要用法
Notion	结合页面、数据库和协作功能的工作空间。	建“课程”页面或数据库 → 添加课程、截止日期和状态等属性 → 关联实验笔记。
Obsidian	基于本地 Markdown 文件的知识管理工具。	创建 Vault (笔记库) → 写笔记 → 用 [[笔记名]] 建内部链接 → 用标签和搜索组织内容。
OneNote	支持文字、手写、图片等的数字笔记软件。	以笔记本-分区-页面组织课程内容，在页面中插入课堂截图或手写记录。
Zotero	文献收集、组织、批注和引用管理工具。	导入题录及 PDF → 检查作者、年份、标题 → 分集合 → 插入报告引用并生成参考文献。
Notion Calendar	用日历安排时间和查看相关事件的工具。	连接支持的日历，安排实验、报告和复习时段。任务列表与日历事件不是同一个概念。
ClickUp	任务与项目协作管理平台。	建列表 → 添加任务、负责人、截止日期和状态 → 用列表或看板追踪。

标签（Tag）用于交叉分类；双向链接帮助查看笔记之间的引用关系；题录/元数据是标题、作者、期刊、年份等描述信息；批注（Annotation）是对原文的高亮或评论；引用样式规定正文和参考文献如何呈现。笔记软件中的“数据库”通常指可按属性查看的结构化条目集合，不要求初学者先写 SQL。

9.2 报告与图表

工具与官方入口	定义	简要用法
Word	文字处理与文档排版软件。	用标题样式组织结构 → 插入图表和题注 → 管理引用 → 导出 PDF 检查版面。
WPS	包括文字、表格和演示等组件的办公套件。	选择课程模板编辑，再检查字体、公式、分页和导出效果。
LaTeX	以源文本描述结构和公式的排版系统。	写 .tex 文件 → 选排版引擎 → 编译 PDF → 根据日志修正错误。
Overleaf	在线 LaTeX 编辑与协作平台。	新建模板项目 → 修改源文件 → 重新编译 → 查看 PDF 和错误日志。
PowerPoint	演示文稿制作软件。	用主题和版式组织幻灯片，插入可编辑框图、曲线和实验照片。

工具与官方入口	定义	简要用法
diagrams.net / draw.io	绘制流程图、系统框图和关系图的工具。	拖放形状 → 添加连接箭头 → 标注 → 保存可编辑源文件并导出图片或 PDF。
PDF 阅读器	软件类别，不是一个特定产品；用于阅读、检索和批注 PDF。	系统自带阅读器即可入门，也可用 Adobe Acrobat Reader 。先看目录、检索关键词，再按页码记录引用。

LaTeX 中文最小示例：推荐在 Overleaf 中选 XeLaTeX；本地可安装 [TeX Live](#) 或 [MiKTeX](#)，使用 VS Code 配合 LaTeX 扩展编辑。发行版提供宏包与引擎，编辑器提供写作界面。

```
LATEX
\documentclass[UTF8]{ctexart}
\begin{document}
\section{RC 低通滤波器}
时间常数为  $\tau=RC$ ，截止频率为
\[
f_c=\frac{1}{2\pi RC}.
\]
\end{document}
```

`\documentclass` 选择文档类；`ctexart` 支持中文文章排版；`\begin` 与 `\end` 界定环境；`\section` 创建章节；`$...$` 是行内数学；`\[...]` 是独立公式；`\frac{a}{b}` 是分式。它应交给 LaTeX 排版引擎，不能用 Python 或 GCC 当成普通程序运行。

图例解释不同曲线的含义；坐标轴标签给出变量与单位；源文件保留可编辑信息。截图可以用于说明，但不能替代原始数据和绘图脚本。

9.3 文件与桌面辅助工具

工具与官方入口	定义	简要用法
7-Zip	压缩与解压软件。	选择文件创建压缩包，或解压课程工程后再打开；不要默认所有工程都能在压缩包内部正常构建。
Everything	Windows 上快速检索文件与文件夹名称的工具。	输入名称或路径片段定位文件，再核对修改时间和目录。默认用途主要是文件名检索。
PowerToys	Microsoft 的 Windows 效率工具集合。	按需要启用窗口布局、批量重命名等模块，先预览批量操作结果。
Snipaste	截图、标注和贴图工具。	截取错误或电路区域 → 用箭头和文字说明 → 保存原图及标注图。
系统截图工具	操作系统内置的截图功能。	Windows 可使用截图工具；macOS 可使用系统截图。适合记录完整报错及其上下文。

10. 终端、远程连接与操作系统

10.1 Terminal、Shell 与 CLI

终端（Terminal）是呈现命令交互的界面；Shell是解释命令的程序；CLI（Command-Line Interface）是命令行交互方式。一个 Windows Terminal 窗口可以打开 PowerShell，也可以打开 WSL 中的 Bash。更换终端外观不会自动更换编译器或 Shell 语法。

工具与官方入口	定义	基本用法
PowerShell	命令解释器与脚本语言，其管道主要传递对象。	用 Get-Location 查看位置、Set-Location 切换目录、Get-ChildItem 列文件，再运行 Git/Python 等命令。
Git Bash	Git for Windows 提供的 Bash 环境及相关工具。	打开后用 pwd、cd、ls 操作目录，并执行 Git。它不等同于完整 Linux 系统。
Bash	Unix 类系统中常见的 Shell 与脚本语言。	交互输入命令，或把命令写成脚本由 Bash 执行。
Windows Terminal	Windows 上管理多标签、配置文件等的终端应用。	新建 PowerShell、命令提示符或 WSL 标签页，配置字体和快捷键。
WezTerm	跨平台终端模拟器与会话相关工具。	创建终端标签/分屏，选择 Shell，需要时配置远程连接。
WSL	Windows Subsystem for Linux，在 Windows 上提供 Linux 环境的技术。	安装发行版后，在其内部运行 Linux 命令和工具；注意 Windows 与 Linux 的路径和软件环境区别。
OpenSSH	SSH 协议的常见实现，提供远程登录等工具。	用 ssh 用户名@主机 连接已有权限的远程机器；用 SFTP/SCP 等传输文件。
Xshell	远程终端客户端。	新建 SSH 会话，填主机、端口和认证方式，再连接。
PuTTY	SSH、串口等连接客户端。	选择连接类型，输入主机或串口信息，配置参数并打开会话。
tmux	终端复用器，可管理会话、窗口和分屏。	tmux new -s lab 创建会话；默认前缀下按 Ctrl+B 再按 D 分离；用 tmux attach -t lab 返回。

SSH（Secure Shell）首先是协议名称，不是只能指某一个软件。tmux 可以让会话在客户端断开后仍保持，但不保证服务器重启、进程崩溃或资源耗尽后任务还能继续。

10.2 操作系统与命令中的基础词

术语	定义与例子
路径 (Path)	文件或目录的位置。绝对路径从根或盘符开始；相对路径依赖当前目录。
当前工作目录	命令当前所处的位置；它会影响相对文件路径和输出位置。
环境变量	程序启动时接收的命名配置值，例如用 <code>PATH</code> 帮助查找可执行程序。
文件权限	决定某个用户能否读取、修改或执行文件等。
进程 (Process)	正在执行的程序实例，具有自己的运行状态和资源。
会话 (Session)	一段持续的交互或运行上下文，具体含义依终端、远程系统或应用而定。
标准输入/输出/错误	程序常见的三个数据通道：输入、正常结果和诊断信息。
管道	把前一个命令的输出交给后一个命令；Bash 与 PowerShell 的数据语义不同。
重定向 > / >>	将输出写入文件；常见用法中 > 会覆盖，>> 会追加。
参数 / 选项	传给程序的信息，例如 <code>git log --oneline</code> 中 <code>--oneline</code> 改变显示方式。
引号	用来控制空格或特殊字符的解释；单引号和双引号的行为依 Shell 而不同。

推荐编辑器与执行器：Shell 脚本可用 VS Code 编辑；`.sh` 常由 Bash 执行，`.ps1` 由 PowerShell 执行。不要把网上某个 Shell 的命令不加区分地粘贴到另一个 Shell。

11. 技术资料、博客与静态网站

11.1 资料类型与检索平台

官方文档解释产品或工具的设计用法；数据手册（Datasheet）给出器件电气参数、引脚和测试条件；参考手册（Reference manual）常详细描述寄存器与外设行为；应用笔记（Application note）围绕具体应用方法提供指导。Errata（补充：勘误）记录已知问题和限制，做芯片项目时也值得查阅。

名称与官方入口	是什么	怎么用
TI	Texas Instruments，半导体厂商，提供器件和应用资料。	以完整器件型号搜索数据手册、参考设计和应用笔记。
Analog Devices / ADI	模拟、混合信号等器件与工具厂商。	查型号资料、信号链应用、电源设计和 LTspice 模型。
ST	STMicroelectronics，意法半导体。	找到目标 STM32 产品页，下载对应数据手册、参考手册和工具。
Espressif	乐鑫科技，ESP 芯片与软件生态提供方。	按具体芯片和 ESP-IDF 版本查文档。
GitHub Discussions	仓库或组织中的社区讨论功能。	搜现有问答、讨论设计思路；明确的软件缺陷可按项目要求提交 Issue。
CSDN / 博客园	中文技术文章与博客平台。	搜具体错误和版本，核对作者环境，再按原文步骤复现。
知乎	问答与文章平台。	用于概念比较和经验了解，技术结论继续追溯原始资料。
掘金	技术内容社区。	通过语言、框架和工具关键词查开发经验。
Stack Overflow	编程问答社区。	用准确报错和语言版本搜索；提问时附最小复现。
Electrical Engineering Stack Exchange	电子工程问答社区。	查询电路、器件和测量问题，附完整电路与参数更容易得到有效回答。
Bilibili / YouTube	视频平台。	观看课程与操作演示，并查配套文件、器件型号和软件版本。
Hackaday	硬件项目与技术内容平台。	看项目思路后追溯源码、原理图和测试记录。
Adafruit / SparkFun	电子产品与动手教程生态。	按模块型号学习接线、供电、驱动安装和示例程序。
Google Scholar	学术文献搜索服务。	搜题目/关键词，查看引用、版本和出版入口。

名称与官方入口	是什么	怎么用
IEEE Xplore	IEEE 等相关出版内容的数字文献平台。	查期刊和会议论文，通过学校授权或开放获取入口阅读全文。
arXiv	预印本存储与传播平台。	查看论文版本和提交日期，再核对是否有正式发表版本。
学校图书馆数据库	学校提供的多种数据库及访问渠道的统称。	通过本校图书馆网站查馆藏和授权资源，并不是一个统一软件品牌。

最小复现是尽量小、但仍能稳定出现问题的例子。预印本是发表流程中的一种公开稿件形态，并不保证已经同行评审。许可证（License）说明软件或资料的使用条件；代码可以公开阅读，不代表没有许可要求。

11.2 博客、静态站与原网站页脚

名词或工具	定义	使用方法或例子
平台博客	在现成平台的编辑器中发布内容。	新建文章、编辑文字图片、设置分类后发布，平台负责大部分网站运行。
静态网站	主要向浏览器提供预先生成的 HTML、CSS、图片等文件。静态站也可以带 JavaScript 交互。	修改源文件，重新生成和发布页面；通常不需要为每次访问动态生成正文。
静态站生成器	把文章源文件、模板与配置转换成静态网站的工具。	写 Markdown，运行生成命令，把输出目录交给托管服务。
Hugo	静态网站生成器。	新建站点并配置主题 → 添加内容 → 本地预览 → 构建发布。
Hexo	基于 Node.js 生态的静态博客框架。	准备 Node.js 环境 → 建博客 → 写文章 → 生成静态文件 → 发布。
Frontmatter	文章正文前的元数据区域。	例如用 title、date、tags 记录标题、日期和标签。
客户端框架	在浏览器侧组织页面和交互的软件框架。	“无客户端框架”不等于网页绝对没有 JavaScript。
Analytics	对访问、事件或使用情况进行统计分析的机制。	页脚写“no analytics”是网站声明，不能仅凭文字独立验证所有网络请求。
Note / Build Note	网站采用的内容类别名称。	通常可译为“笔记”和“构建/开发记录”，不是统一行业认证。

一个 frontmatter 示例：

```
YAML
---
title: RC 滤波器实验
date: 2026-09-22
tags:
  - 电路
  - Python
---
```

其中冒号连接键和值，缩进和连字符表达列表。这是给网站生成器读取的数据，不会由浏览器直接当成网页结构显示。

11.3 相关文章标题中的名词

- HarmonyOS：华为的操作系统与应用生态相关名称；文中仅出现原型文章标题，无法据此判断其针对的版本或设备。[HarmonyOS 开发者入口](#)
- Prototype（原型）：用来验证某些交互、技术或需求假设的早期实现。原型运行成功不自动证明生产级兼容性、性能或可靠性。
- Persistent terminal（持久终端）：通常指在多次连接或操作之间保存运行上下文的终端机制。能保存哪些状态，要看具体实现。
- Shared（共享）/ Observer（观察者）：可能表示多个参与方查看或操作同一运行状态；标题本身没有说明并发控制方式。
- Broker（中介服务）：在客户端和底层资源之间协调消息、连接或请求的组件。具体持久终端 broker 的职责须读原文才能确认。
- Architecture（架构）：系统组件、责任分工及交互关系的设计；不等于界面截图或文件列表。

这些解释是通用词义，不是对未提供的四篇相关文章实现细节的推断。

12. AI 辅助学习和编程

12.1 先理解名词，而不是先比较品牌

术语	定义与例子
AI	Artificial Intelligence，人工智能，涵盖多种方法与系统。原笔记主要讨论生成式 AI 工具。
模型 (Model)	经过训练、根据输入产生预测或输出的计算系统。模型不等于承载它的聊天界面或编辑器。
大语言模型 (LLM ，补充)	主要处理语言序列的模型；现代产品可能还结合视觉、音频等能力。
对话助手	通过自然语言交互帮助解释、写作或分析的产品形态。是否能执行代码、检索或访问文件取决于工具和权限。
AI 编辑器	将补全、问答或工程修改能力整合进代码编辑界面的软件。
Coding Agent	能围绕编程任务循环读取上下文、调用工具、修改文件和检查结果的系统。实际操作能力取决于配置。
Agent 运行框架 / Harness	连接模型、上下文、工具、权限和执行循环的软件层。Pi 属于这一类；框架本身不等于模型。
Vibe Coding	一种以自然语言描述需求、运行生成结果并反复反馈的开发方式。它不是语言、编译器或正式验证方法。
Prompt / 提示词	给模型的任务说明、上下文和输出要求。输入、限制、成功标准写得具体，结果更容易检查。
上下文 (Context)	模型当前可以使用的信息，如对话、工程文件、文档和工具结果。未提供或未读取的材料不应假定模型已经知道。
多模态 (Multimodal)	能处理文字、图像、音频等不同形式的信息。具体产品是否支持每一种输入需另查。
工具调用	模型通过系统提供的接口请求检索、读文件或执行程序等操作。生成“我已经测试”这句话不等于发生了工具调用。
本地运行 / 云端运行	区分计算和数据处理发生的位置。编辑器安装在本地，不意味着模型推理也全部在本地。
扩展 / 插件	为应用增加功能的模块，可能连接模型、文件或外部服务。
配额 / 额度	对调用次数、用量或资源的限制，不等于某个固定的能力等级。
代码审查 (Code review)	检查修改是否正确、易维护并符合项目要求；常结合 diff、测试和运行结果。

术语	定义与例子
测试 (Test)	通过输入、执行和判断条件检查程序行为。测试覆盖不到的情况仍可能有错误。

12.2 学习、阅读和检索工具

工具与官方入口	定义	初学者如何使用
ChatGPT 官方学习入口	OpenAI 的 AI 产品，可通过对话解释、分析和完成支持的任务。	提供题目和自己的推导，要求逐步解释变量、单位与假设；计算题要求给出可检查过程。 官方产品说明
Claude	Anthropic 的 AI 助手与产品生态。	提供文档或代码，要求提炼结构、指出证据和解释错误；不要把 Claude 对话产品与 Claude Code 完全等同。
Gemini	Google 的 AI 助手及模型品牌相关产品。	通过对话处理题目、文字或产品支持的其他输入；开发者接口实验可使用 AI Studio。
DeepSeek	深度求索提供的模型与相关服务。	输入概念或代码问题，要求给出推导、边界条件和可复核的结果。
NotebookLM 原官网入口	原笔记所指的围绕用户提供资料组织问答与学习的工具。	加入课程资料 → 提出具体问题 → 跟随引用回到原文核对。本次原入口跳转到 notebook.google ，应以当前官方界面名称为准。
Perplexity	将问答与资料检索结合的工具。	搜具体问题，打开给出的来源，确认来源真的支持该结论。
Google AI Studio	面向 Gemini 的提示词试验和开发原型环境。	选择支持的模型与输入方式 → 调整提示词和参数 → 观察结果 → 按文档转为 API 调用。
Ollama	运行和调用模型的工具生态，包含本地模型运行能力。	选择适合硬件的模型，下载并运行，再通过界面或接口交互。核对实际采用的是本地还是云端模型，以及外部工具是否联网。

如何问得更有效：与其只问“解释 RC”，可以写：“我是刚学电路的一年级学生，请从电容电流关系推导 RC 低通的时间常数，解释单位，并用 10 kΩ、100 nF 算一个例子；说明理想输入源和负载假设。”

12.3 AI 编辑器与 Coding Agent

这些产品形态可能重叠，一个品牌也可能提供编辑器、终端和网页入口。下面按原笔记的学习用途讲解，不做价格或性能排名。

工具与官方入口	定义	基本用法
GitHub Copilot	GitHub 的 AI 编程助手，提供代码辅助及相关工作流。	在受支持环境登录 → 打开工程 → 让其解释选中代码或提出修改 → 审查并运行结果。

工具与官方入口	定义	基本用法
Cursor	将 AI 工程理解与代码修改整合到开发工作流中的产品。	打开项目 → 提供任务和相关文件 → 检查多文件修改 → 运行测试。
Windsurf 原官网	原笔记中的 AI 开发环境名称。	基本流程同样是打开项目、给出任务、查看修改并验证。本次原网址跳转到 Devin Desktop ，当前产品界面与旧教程可能不同。
Google Antigravity IDE	Google Antigravity 产品体系中的 Agent 开发环境。	选择项目 → 描述任务 → 让 Agent 使用编辑器、终端等工具 → 查看产物与验证结果。Antigravity 现在有多种产品形态，不能仅凭品牌名认定是同一个 IDE。
TRAE	原笔记所指的 AI 开发产品；当前官网分别列出 TraeCode 与 TraeWork。	编程任务应选对应代码工作流 → 打开工程 → 指定任务 → 查看修改与运行结果。不要把工作助手入口当成固定的旧版编辑器界面。
Claude Code	可读取工程、编辑文件和运行命令的 Agent 编程工具。官方提供终端及其他入口。	在工程中启动或打开对应入口 → 说明目标 → 检查 diff、构建输出和测试记录。它不只限于终端。
OpenAI Codex 官方文档	面向理解代码、实现、调试、测试和审查等任务的编程 Agent 产品。	选择项目和环境 → 给出输入、约束与验收条件 → 检查修改及实际执行的验证。它不是编译器；仍需调用项目自己的工具链。 官方学习入口
Pi	轻量、可扩展的终端 Coding Agent 运行框架。此处不是 Raspberry Pi 单板计算机，也不是同名聊天产品。	按官方文档配置模型访问 → 在项目目录启动 → 给出任务 → 查看工具调用和修改；可通过扩展等定制工作方式。

Pi 的歧义说明：原文未保留其软件包链接。本表依据“终端 Agent、可扩展运行框架”的描述，对应到 pi.dev 的 Pi；若原站链接指向另一个同名项目，应以原链接为准。

12.4 Web 原型工具及相关名词

工具与官方入口	定义	基本用法
v0	Vercel 的 AI 网页/应用构建工具。	描述页面、布局和交互 → 预览生成结果 → 修改需求 → 检查代码和部署设置。
Bolt.new	在浏览器中进行 AI 应用构建的工具。	输入功能需求 → 查看运行预览 → 迭代 → 检查依赖、数据连接与输出。
Lovable	通过自然语言构建网页和应用的工具。	描述页面及流程 → 连接需要的数据服务 → 验证登录、访问控制和实际交互 → 发布。

术语	定义
Web / 前端 (Frontend)	用户通过浏览器看到和交互的部分，包括界面、样式与交互逻辑。
组件 (Component)	可复用的界面或功能单元，例如按钮、表格、导航栏。
响应式布局	页面根据屏幕尺寸调整排列、间距和可见内容。
技术栈 (Tech stack)	项目所组合使用的语言、框架、数据库与部署工具等。
认证 (Authentication)	确认“你是谁”，例如登录。
授权 / 权限 (Authorization)	确认“你可以做什么”，例如是否能查看其他人的实验数据。
部署 (Deployment)	把程序和配置放到目标运行环境并使其可使用。
代码导出	获取生成的源代码和相关配置，以便继续维护；能导出代码不等于所有外部服务配置也已齐全。

这些工具适合课程资料页、数据看板和作品展示。它们生成的网页原型不能直接替代电路模型、PCB 制造检查、MCU 调试或 FPGA 时序分析。

12.5 把任务描述写成可验收的要求

原笔记中的 CSV 任务已经包含四类关键内容：

- 1. 输入：文件在哪里、字段叫什么、单位是什么。
- 2. 输出：生成什么文件、放在哪里、图上应有什么。
- 3. 约束：允许修改哪些文件、不得改变哪些数据。
- 4. 验收：执行了什么、结果是什么、哪些步骤没有验证。

其中“只修改指定脚本”是文件范围约束；“检查缺失值和时间顺序”是功能要求；“不安装软件或上传数据”是操作约束；“未执行测试单独列出”是证据要求。编写任务时把这些类别讲清楚，通常比单纯要求“做得专业”更有帮助。

13. 一套最小学习组合与综合练习

13.1 按任务选择，不按软件数量选择

当前任务	可采用的起步组合	先掌握什么
学 C/C++ 基础	VS Code + 本机 GCC/Clang；或课程指定 IDE	编译、运行、错误信息、断点、变量。
做数据处理	Python + VS Code/PyCharm + venv + pandas/Matplotlib	解释器选择、CSV 字段、单位、绘图。
学模拟电路	LTspice；若课程用 Multisim 则沿用	原理图、参考地、模型、工作点、瞬态、AC。
学 PCB	嘉立创 EDA 或 KiCad 二选一入门	符号、封装、引脚映射、ERC/DRC、制造输出。
学 STM32	课程开发板 + STM32CubeIDE + 匹配探针	芯片型号、引脚、电平、构建、烧录、调试。
学 ESP32	对应型号的 ESP-IDF 工程或课程框架	目标芯片、工具链、串口、SDK 版本。
学 FPGA	课程指定 Quartus/Vivado + 仿真器	模块、testbench、时钟复位、约束、时序报告。
写报告与查文献	Word 或 LaTeX + Zotero	图表单位、引用、版本、原始资料来源。
管理学习成果	Git + README + 一种笔记工具	目录、提交、复现步骤与数据备份。

这里的选择是面向入门任务的建议，不表示这些产品在所有情境下都优于其他产品。先服从课程器件与版本要求，再优化个人习惯。

13.2 用 RC 练习把术语串起来

任务：验证 $10\text{ k}\Omega$ 、 100 nF 一阶低通的时间常数和截止频率。

- 理论计算：用纸笔或第 1 节的 Python 例子，算出 $\tau=1\text{ ms}$ 、 $f_c\approx 159.15\text{ Hz}$ 。
- 仿真工程：用 LTspice 建原理图，确认输出取在电容两端，并设置正确参考地。
- 瞬态分析：输入阶跃，测出响应到达最终变化量约 63.2% 的时间；这个时间应接近一个 τ 。
- AC 分析：绘制 V_{out}/V_{in} ，寻找相对低频幅值下降约 3.01 dB 的频率。
- 可选实物实验：先核对仪器接地、幅度、器件和 ADC 范围，再采集实际数据；记录元件容差、探头和负载条件。
- 数据后处理：保存原始 CSV，在副本或脚本流程中检查时间与单位，绘制理论、仿真及实测对比。
- 版本管理：提交脚本、参数和报告，使用 `.gitignore` 排除可再生缓存；原始数据另有备份。
- 报告表达：写清“理论值是多少、仿真/实测是多少、差异可能来自哪里、哪些环节尚未验证”。

推荐目录：

TEXT

```
rc-filter/  
├─ README.md      项目目标、环境、运行步骤  
├─ circuit/       电路工程、模型引用与参数  
├─ data/          原始记录及采样条件说明  
├─ scripts/       数据处理源代码  
├─ figures/       生成的结果图  
└─ report.md      推导、结果和误差讨论
```

├─ 与 └─ 只是目录树的展示符号，不是必须输入的命令。目录后的说明文字也不是文件名的一部分。

13.3 最值得先记住的十个区别

1. 编辑器与编译器：一个写代码，一个翻译代码。
2. Python 与 PyCharm：一个是语言及相关运行实现，一个是开发环境。
3. Git 与 GitHub：一个管理版本，一个提供托管与协作。
4. 提交与推送：一个创建本地版本，一个同步到远端。
5. 终端与 Shell：一个呈现交互，一个解释命令。
6. 原理图与 PCB：一个表达电气连接，一个安排实体板上的实现。
7. 仿真与实测：一个求解模型，一个观察真实系统。
8. HDL 仿真与综合：一个检查模型行为，一个生成逻辑结构。
9. TCAD 与 SPICE：前者深入工艺/器件物理，后者常在电路层面连接器件模型求解。
10. AI 模型与 Agent：模型生成判断和输出，Agent 系统围绕任务组织模型、上下文和工具执行。

来源、范围与核对说明

- 术语范围依据用户提供的《微电子学习工具笔记》全文；少量补充术语已标明或放在相关解释中。
- 工具信息优先参考文中链接的厂商官网、官方文档和开源项目维护者页面；正文中的操作步骤是教学性概述，不是逐字摘录。
- 本次已查阅主要开发、仿真、数据处理和 AI 工具的官方入口。部分页面只返回简略信息；另有 GDB、Make、J-Link GDB Server、PuTTY、HSPICE 等页面未能通过当前读取工具获取正文，其链接作为官方查阅入口，本文不声称已逐一验证所有页面或下载入口可用。
- Windsurf、NotebookLM 的原网址在本次查询中发生跳转；本文保留原名称并注明观察到的入口变化，不据此推断所有版本的迁移情况。
- Pi 的具体项目对应关系依据原文描述推定，原文未保留软件包链接；其他相关文章也只按已提供标题解释术语。
- 产品价格、账号权益、地区可用性、具体硬件支持表和安装版本不属于本指南的逐项核验范围。
- 已实际运行第 1 节 Python 的 RC 计算示例，输出时间常数 1.000 ms、截止频率 159.15 Hz；并核对了文档表格结构和原笔记所列工具的覆盖情况。
- 其余代码和网表用于讲解。C/C++、LaTeX 示例未在本次运行；未进行真实硬件、商业 EDA、SPICE 或 HDL 仿真实验；演示数据不得作为实测数据引用。